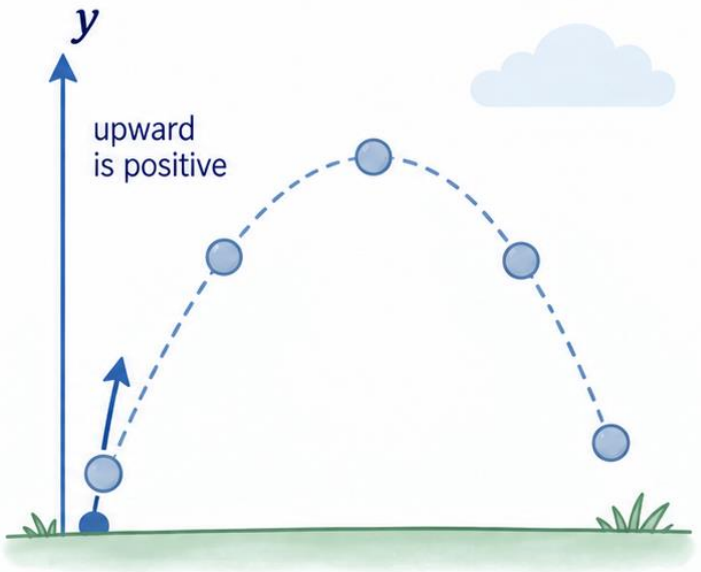
Numerical Algorithms:
Turn physical reasoning into pseudocode, loops, and fixes



- A familiar equation can be evaluated one time at a time.
- An algorithm makes the order visible.
- A loop makes repetition explicit.
- Evidence still needs checking and physical interpretation.

Question
Start from a physical prediction

→

Recipe
Write the algorithm as pseudocode

→

Loop
Repeat the calculation in a `for loop`

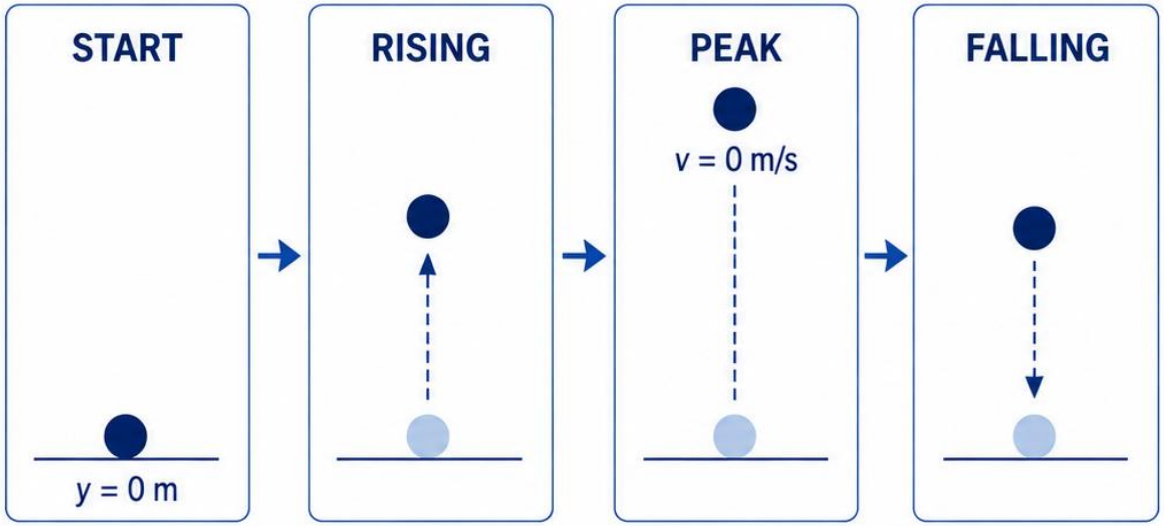
→

Evidence
Compare results and check for defects

→

Explanation
Confirm and give a physical interpretation

Predict the motion before asking MATLAB to repeat anything



1. Choose upward as positive.
2. The ball starts at y = 0 m.
3. It rises to one maximum, then falls.
4. The prediction is our first debugging reference.



What should the output look like?

Fix the model, then ask what changes

Vertical launch model

$$y(t) = y_0 + v_0 t - \frac{1}{2} g t^2$$

Fixed physical inputs (stay the same for every evaluation)

$y_0 = 0\ \text{m}$
initial position

$v_0 = 20\ \text{m s}^{-1}$
initial velocity (upward is positive)

$g = 9.81\ \text{m s}^{-2}$
gravity (constant)

Ordered times

$t_s = 0{:}0.5{:}4\ \text{s}$
use the same time grid for all calculations

One evaluation gives one position

`current_time_s`
a single time value from `t_s`

→

$y(t) = y_0 + v_0 t - \frac{1}{2} g t^2$
use the fixed inputs

→

$y(t)$
one position (in metres)

Only `current_time_s` changes; the model and inputs stay fixed.

One calculation is four small jobs

Before MATLAB syntax, make the order of reasoning visible.

INPUT
store the model parameters and ordered times

→

CALCULATE
use the current time in the position equation

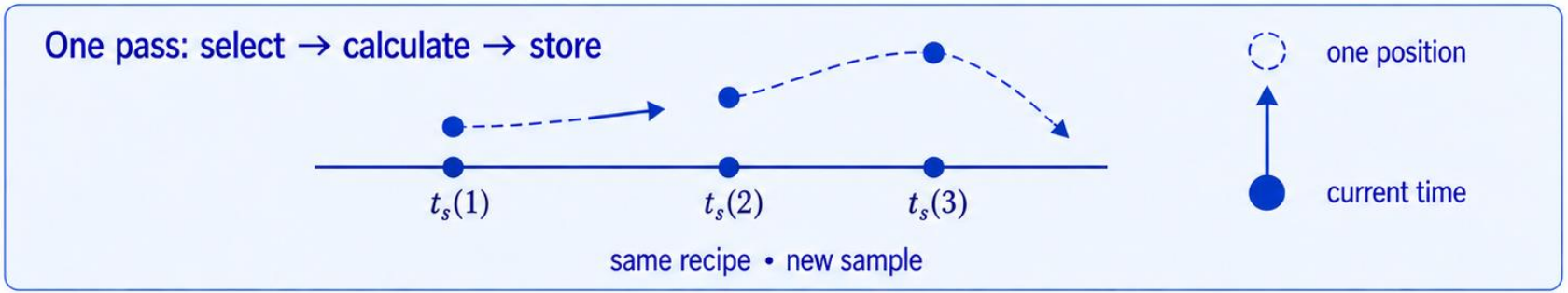
→

STORE
write one result to the matching output slot

→

CHECK
compare with something already known

One pass = one time → one position



An index traces both selection and storage

MATLAB indexing starts at 1.

t_s(sample_index)	0	0.5	1.0	1.5	⋯
sample_index	1	2	3	4	⋯

y_m(sample_index)	0	8.77375	⋯
sample_index	1	2	⋯

Pass 1

index	index 1
current time	$t = 0$ s
calculation	$y =$ (using t_s(sample_index))
stored result	$y = 0$ m

Same sample_index

Pass 2

index	index 2
current time	$t = 0.5$ s
calculation	$y =$ (using t_s(sample_index))
stored result	$y = 8.77375$ m

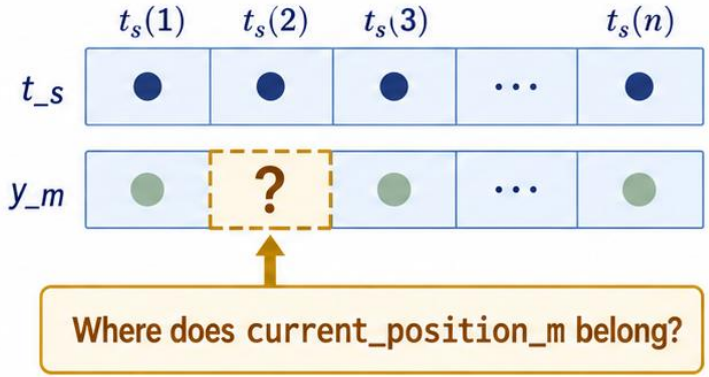
Notice the changing quantity:
the `sample_index` moves to the next element, so both the time and stored result change.

Calculate one value; store it in the matching slot

```
n_samples = numel(t_s);
y_m = zeros(size(t_s));

for sample_index = 1:n_samples
   current_time_s = t_s(sample_index);
   current_position_m = y0_m + v0_mps*current_time_s ...
          - 0.5*g_mps2*current_time_s^2;
   % missing storage statement
end
```

1. Preallocate one output slot per time.
2. Loop from 1 to `n_samples`.
3. Read `current_time_s = t_s(sample_index)`.
4. Calculate `current_position_m` from the scalar time; then ask where it belongs.

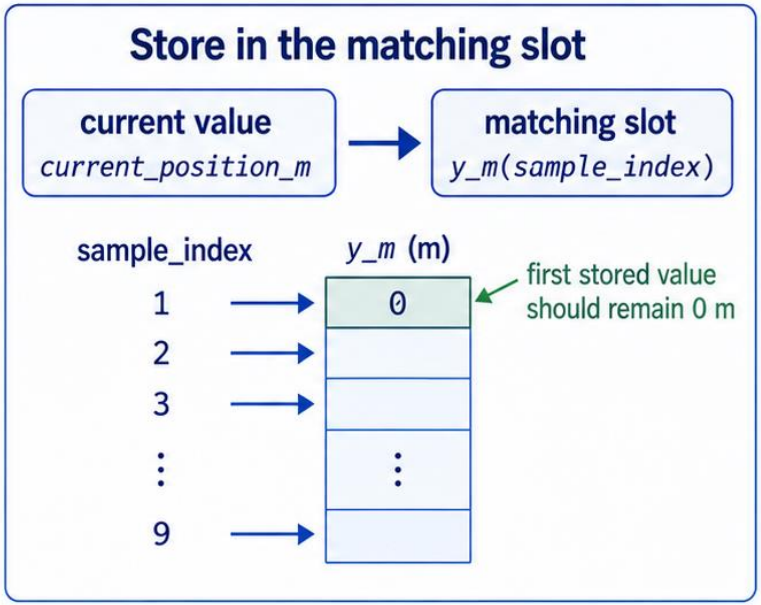


Your turn: complete the storage line

Use the loop to store each position in the matching output slot.

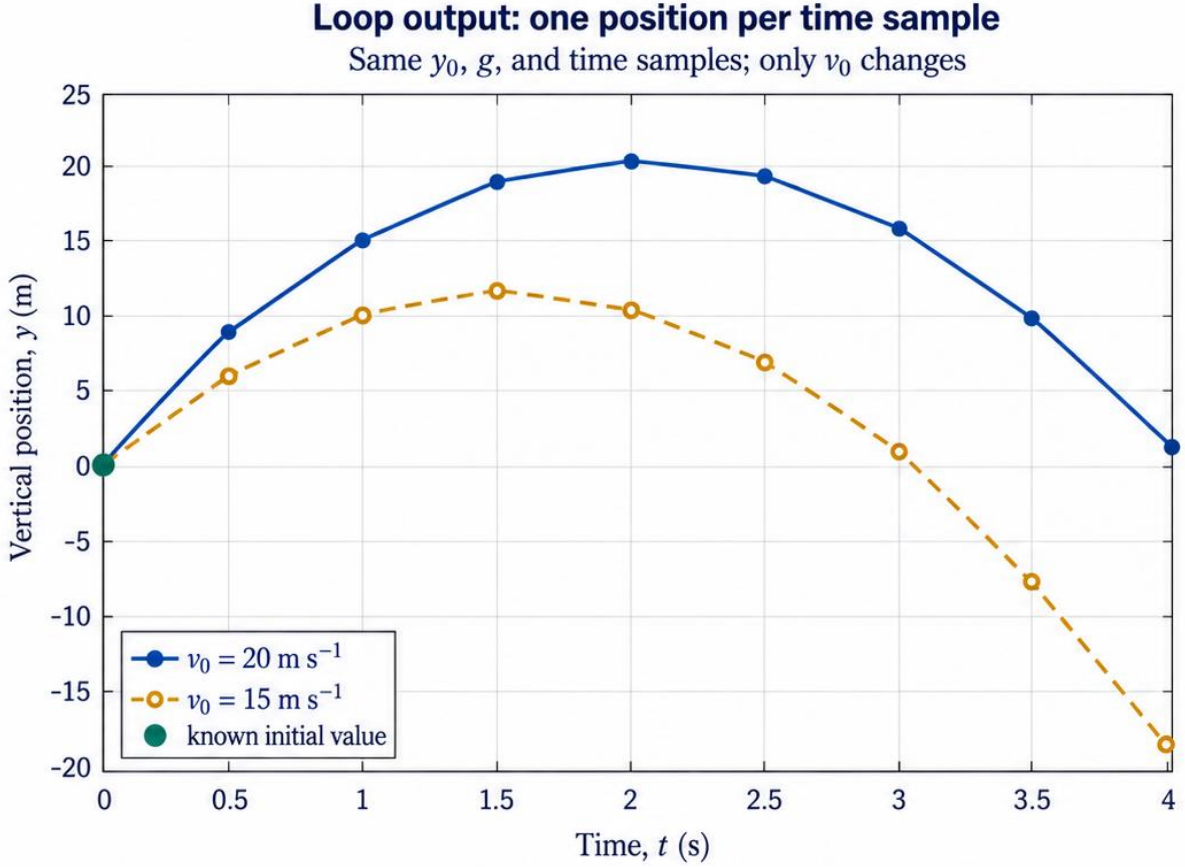
```
% Vertical launch: store position at each time
y0_m    = 0;
v0_mps  = 20;
g_mps2  = 9.81;
t_s     = 0:0.5:4;
y_m     = zeros(size(t_s));

for sample_index = 1:length(t_s)
   current_time_s = t_s(sample_index);
   current_position_m = y0_m + v0_mps*current_time_s ...
                        - 0.5*g_mps2*current_time_s^2;
   y_m(sample_index) = _______;
end
```



- The blank must store the current scalar result.
- It must use the current index.
- It must not overwrite one fixed element.
- The first stored value should remain 0 m.

Change one input, predict one consequence



Fixed
- y_0 (m)
- g (m s^{-2})
- $t_s = 0{:}0.5{:}4$ s

Changed
- v_0 (m s^{-1}) $20 \rightarrow 15$

Predicted
- lower sampled peak

20 m s^{-1}: peak position 20.38 m at 2.0 s
Final sampled position 1.52 m at 4 s

15 m s^{-1}: peak position 11.46375 m at 1.5 s
Final sampled position −18.48 m at 4 s

The lower launch speed has a lower sampled peak.

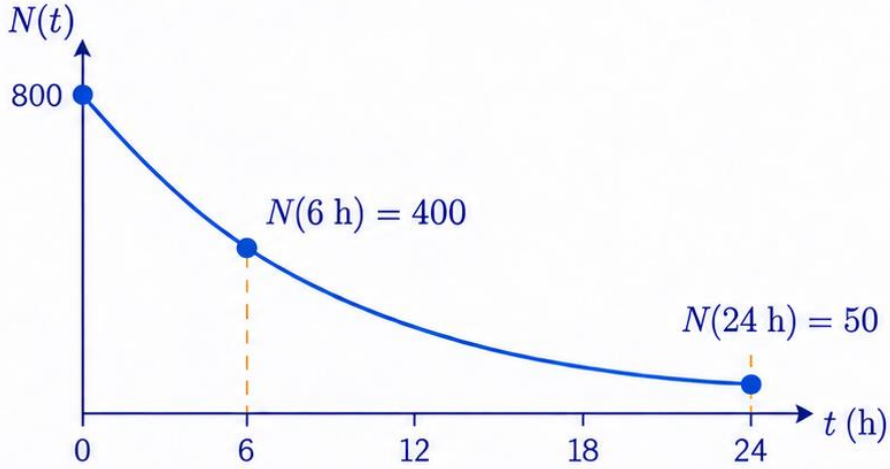
No-impact model: the calculation continues to 4 s. $y = 0$ is the launch reference, not an automatic stopping rule.

A second process can use the same loop logic

Radioactive decay follows the same input → current value → calculate → store → check pattern.

$$N(t) = N_0\, e^{-\lambda t}, \qquad \lambda = \frac{\ln 2}{T_{1/2}}$$

$N(t)$	number of counts at time t
N_0	initial number of counts
λ	decay constant
$T_{1/2}$	half-life



Fixed inputs

```
N0_counts = 800;
half_life_h = 6.0;
t_decay_h = 0:1:24;
```

→

Run the same loop logic

Use the current time to calculate $N(t)$, store one count, and check against known values.

→

Check (known values)

- Expect a decreasing count
- $N(6\ \text{h}) = 400$
- $N(24\ \text{h}) = 50$

Transfer challenge — write the loop in plain language first

Input the count, half-life, decay constant, and hourly times.
For each index, select the current time. Calculate one count and store it in the matching output slot.
Check the half-life point and stop after the final hour.

Pseudocode (plain language)

1 Input

Store the count, half-life, decay constant, and hourly times.
$N_0 = 800$, half_life_$h = 6.0$, $\lambda = \log(2)/\text{half_life_}h$,
$t_h = 0{:}1{:}24$.

2 Select

For each index, select the current time.
Let current_time_h be the time at this index.

3 Calculate

Use the decay model to calculate one count.
current_count = N_0 $\exp(-\lambda$ current_time_$h)$.

4 Store

Save the count in the matching output slot.
Place current_count in _____ .

5 Check

Check the half-life point (6 h) and stop after the final hour.
After the last time, the loop is complete.

Transfer from the vertical model

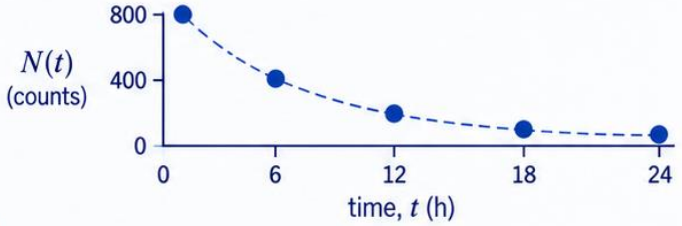
The loop pattern is the same as for position:
input → calculate → store → check, now applied to count $N(t)$.

Radioactive decay model

$$N(t) = N_0 \exp(-\lambda t)$$

$N_0 = 800$ counts
half_life_h = 6.0 h
$\lambda = \log(2)/\text{half_life_h}$
$t_h = 0{:}1{:}24$

Decreasing count (sketch)



? What should be true at 6 h?

Use the model and values above to state the expected count at 6 h.

Diagnose the defect before editing the code

- Syntax defects stop MATLAB reading the intended structure.
- Indexing or operator defects can run while selecting or storing the wrong thing.
- Physical or logical defects can produce plausible numbers with wrong physics.
- The code ran is not a correctness test.

SYNTAX

stops MATLAB reading the intended structure

```
for i = 1:length(t_s)
   y_m(i) = y0 + v0*t_s(i) - 0.5*g*t_s(i)^2
% missing end
```

Symptom	→	MATLAB does not run the code.
Evidence	→	"end expected"
Repair cue	→	Add the missing **end** in the correct place.

INDEXING / OPERATOR

can run while selecting or storing the wrong thing

```
y_m(i) = y0 + v0*t_s(i) - 0.5*g*t_s^2
% should be t_s.^2
```

Symptom	→	Code runs but gives unexpected values.
Evidence	→	Dimension mismatch or a result that does not match a known value.
Repair cue	→	Use the correct operator: `t_s.^2` (element-wise).

PHYSICAL / LOGICAL

can produce plausible numbers with wrong physics

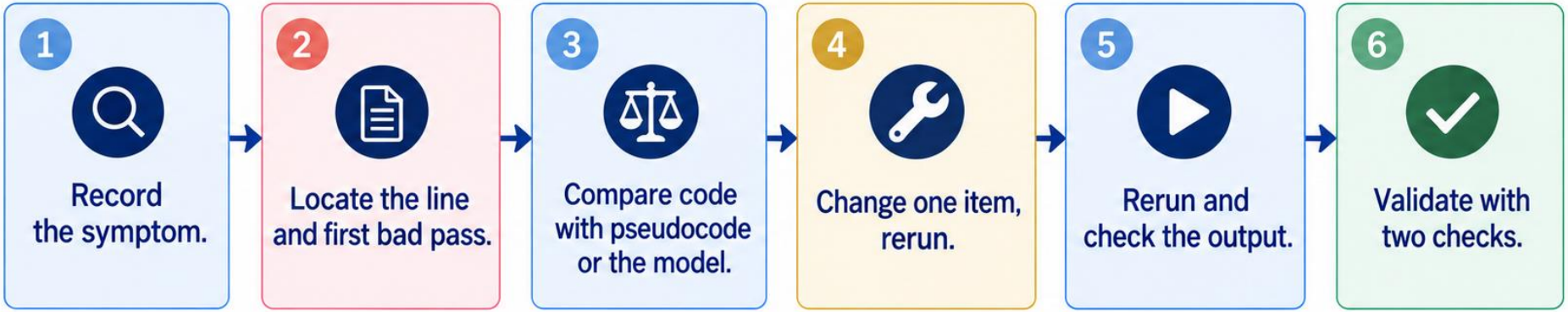
```
y_m(i) = y0 + v0*t_s(i) - 0.5*(-g)*t_s(i)^2
% gravity sign is wrong
```

Symptom	→	Code runs and produces plausible numbers.
Evidence	→	Position increases without reaching a maximum (for $v_0 = 20\ \mathrm{m\ s^{-1}}$).
Repair cue	→	Use $g = 9.81\ \mathrm{m\ s^{-2}}$ with a negative sign: `- 0.5*g*t_s(i)^2`.

The code ran is not a correctness test. Check the output against a known value and the physical behaviour.

Use evidence to locate the first bad pass

Turn the defect categories into a repeatable evidence-based repair habit.



One controlled change keeps the evidence interpretable.

Confirm the fix with numerical and physical evidence

Repair evidence must agree with both the computation and the physics.

First bad pass

i	$t_s(i)$ [s]	$y_m(i)$ [m]
1	0.0	0.00000
2	0.5	11.22625
3	1.0	24.90500

Compare code with the model

Correct (matches the model)

```
y_m(i) = y0_m + v0_mps*t_s(i) -
         0.5*g_mps2*t_s(i)^2;
```

Common defect (wrong sign)

```
y_m(i) = y0_m + v0_mps*t_s(i) +
         0.5*g_mps2*t_s(i)^2;
```

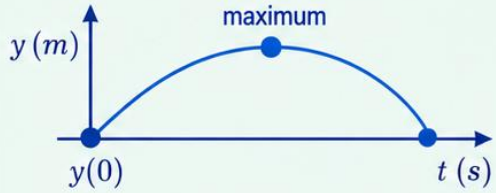
Two checks

Known value

$y(0.5\ \text{s}) = 8.77375\ \text{m}$

Physical behaviour

rises to one maximum, then falls



A passing check can still miss wrong physics

- y(0) = 0 m is a known initial-value check.
- assert(y_m(1) == y0_m) checks the first stored value.
- A wrong gravity sign can still pass at $t = 0$.
- Compare direction and shape with the physical prediction: check the number and the meaning.

1 Model check

Known initial value

$$y(t) = y_0 + v_0 t - \frac{1}{2} g t^2$$

$$y_0 = 0 \text{ m} \quad v_0 = 20 \text{ m s}^{-1}$$

$$g = 9.81 \text{ m s}^{-2}$$

$y(0) = 0$ m
is a known initial-value check.

2 Code / output check

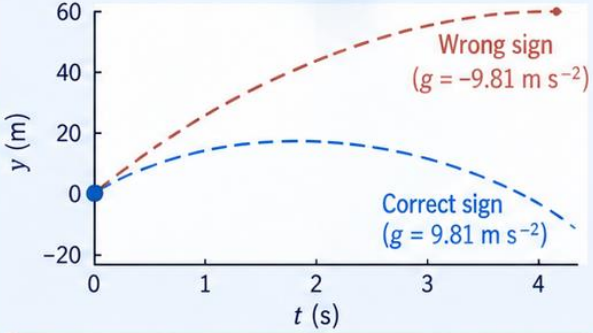
First stored value

```
t_s = 0:0.5:4;
y_m = zeros(size(t_s));
assert(y_m(1) == y0_m)
% checks the first stored value
```

Pass
The first stored value is correct (y_m(1) = 0 m).

3 Physical meaning check

Direction and shape



Wrong gravity sign can still pass at $t = 0$.
The trajectory looks plausible, but it is physically inconsistent.

Check the number and the meaning.
A passing check confirms one part of correctness, but physical interpretation is still required.

Exit ticket — explain the loop to a future you

Write a brief, clear explanation as if you will read it again next month.

Order

Put input, calculation, output, and check in order.

Trace

Explain what *t*_s(sample_index) selects.

Diagnose

Identify one defect and its evidence.

Validate

State one validation and one physical conclusion.

These ideas lead to next week:
a two-equation circuit recipe.

$$(R_1 + R_s)\,I_1 - R_s\,I_2 = V_1$$
$$-R_s\,I_1 + (R_2 + R_s)\,I_2 = V_2$$

Week 03 preview